

Guide d'utilisation

Installer, utiliser et étendre portopt

Ce guide explique comment se servir de la librairie : installation, conventions, chaque module avec ses paramètres et ses sorties, des recettes pour les usages courants, et les erreurs fréquentes. Tous les exemples ont été exécutés; les sorties affichées sont réelles (lignes `dtype omises`) et utilisent l'univers simulé `simulate_returns()`. La théorie des méthodes est dans le document compagnon *Allocation de portefeuille risk-based* (`docs/methodology.pdf`).

Average capital allocation										
Mean target weight across rebalancings										
	SPY	EFA	EEM	IWM	TLT	IEF	LQD	GLD	DBC	VNQ
1/N	10%	10%	10%	10%	10%	10%	10%	10%	10%	10%
InvVol	8%	8%	6%	6%	10%	21%	19%	9%	7%	6%
RB bond tilt	6%	6%	4%	5%	14%	29%	18%	7%	7%	4%
ERC	7%	7%	5%	6%	13%	26%	14%	9%	8%	5%
ERC LW	7%	7%	5%	6%	14%	24%	14%	10%	8%	5%
HRP	3%	4%	3%	2%	6%	39%	28%	7%	4%	3%

Table des matières

1	Installation	2
2	Conventions	2
3	Module data	2
4	Module estimator	4
5	Module backtester	7
6	Module metric	8
7	Module plot	10
8	Recettes	11
9	Erreurs fréquentes	12
10	Référence de l'API	13

1 Installation

portopt demande Python 3.10 ou plus. Depuis la racine du dépôt :

```
pip install -e .           # numpy, pandas, scipy, yfinance
pip install -e ".[plot]"  # + matplotlib, pour le module plot
pip install -e ".[dev]"   # + matplotlib et pytest
pytest                    # 42 tests, quelques secondes
```

Le dépôt est organisé ainsi :

```
portopt/
  data.py           téléchargement, nettoyage, rendements, univers simulé
  estimator.py      règles d'allocation et estimateurs de covariance
  backtester.py     backtest walk-forward
  metric.py         indicateurs de performance et de risque
  plot.py           graphiques et rapport PNG
  fonts/           polices utilisées par plot (licence OFL)
  examples/run_backtest.py  comparaison complète en ligne de commande
  docs/            méthodologie, ce guide, figures
  tests/           tests unitaires
```

Pour vérifier l'installation sans réseau :

```
from portopt.data import simulate_returns
from portopt import ERC, run_backtests, summary

rets, _ = run_backtests(simulate_returns(), {"ERC": ERC()}, 252, 21)
print(summary(rets).loc["Sharpe"].round(3))
```

```
ERC      0.776
Name: Sharpe, dtype: float64
```

2 Conventions

La librairie repose sur quelques conventions. Les connaître évite la plupart des erreurs.

Objet	Convention
Rendements	<code>DataFrame</code> indexé par un <code>DatetimeIndex</code> (dates en lignes), une colonne par actif. Rendements simples , pas logarithmiques.
Date	La ligne t contient le rendement de la clôture $t - 1$ à la clôture t .
Valeur manquante	<code>NaN</code> signifie « actif non disponible ». Le backtester n'investit que dans les actifs sans <code>NaN</code> sur la fenêtre d'estimation.
Poids	<code>pd.Series</code> indexée par actif, positive, de somme 1.
Fréquence	Quotidienne par défaut (<code>periods_per_year=252</code>). Pour d'autres fréquences, passer <code>periods_per_year</code> aux fonctions de <code>metric</code> .
Taux sans risque	<code>sharpe_ratio(rf=...)</code> : taux annuel .
Seuil du Sortino	<code>sortino_ratio(target=...)</code> : seuil par période .
Seuil du PSR	<code>probabilistic_sharpe_ratio(sr_benchmark=...)</code> : Sharpe annualisé .
Coûts	<code>transaction_cost_bps</code> en points de base, appliqués au turnover one-way.

3 Module data

3.1 Obtenir des rendements

`load_returns` enchaîne téléchargement, nettoyage et calcul des rendements :

```
from portopt import load_returns

R = load_returns(["SPY", "TLT", "GLD"], start="2010-01-01", end="2024-12-31",
                 cache_dir="data_cache")
```

Avec `cache_dir`, le premier appel enregistre les prix en CSV et les appels suivants les relisent sans réseau. Le nom du fichier dépend des tickers, des dates et du champ demandé : changer un paramètre déclenche un nouveau téléchargement.

Fonction	Rôle et paramètres
<code>download_prices</code>	Prix Yahoo Finance. <code>tickers</code> , <code>start</code> , <code>end</code> (exclue), <code>field="Close"</code> , <code>auto_adjust=True</code> (prix ajustés des dividendes et splits), <code>cache_dir</code> .
<code>clean_prices</code>	<code>ffill_limit=5</code> : comble les trous d'au plus 5 jours avec la dernière valeur. <code>min_history</code> : retire les actifs ayant moins d'observations. Jamais de remplissage avec une valeur future.
<code>compute_returns</code>	<code>method="simple"</code> ou <code>"log"</code> , <code>period=1</code> . Retire les premières lignes sans rendement.
<code>resample_returns</code>	Change de fréquence en composant les rendements : <code>rule="W-FRI"</code> , <code>"ME"</code> , etc.
<code>load_returns</code>	Les trois étapes ci-dessus, en rendements simples.
<code>simulate_returns</code>	Univers simulé de 10 ETF (SPY, EFA, EEM, IWM, TLT, IEF, LQD, GLD, DBC, VNQ) : <code>n_obs=3780</code> , <code>start</code> , <code>seed=15</code> .

3.2 Travailler hors ligne

```
from portopt.data import simulate_returns

R = simulate_returns()
print(R.shape, R.index[0].date(), R.index[-1].date())
print(list(R.columns))

(3780, 10) 2010-01-01 2024-06-27
['SPY', 'EFA', 'EEM', 'IWM', 'TLT', 'IEF', 'LQD', 'GLD', 'DBC', 'VNQ']
```

Le simulateur reproduit des corrélations réalistes, des régimes de stress et des queues épaisses. Il sert aux tests, aux exemples et à la documentation.

3.3 Nettoyer ses propres prix

Si vous avez vos propres prix (fichier CSV, base interne), passez-les à `clean_prices` puis `compute_returns`. L'exemple ci-dessous simule un actif coté plus tard (SPY) et un trou de trois jours (EFA) :

```
import numpy as np
from portopt.data import clean_prices, compute_returns

prices = 100 * (1 + R).cumprod()
prices.iloc[:300, 0] = np.nan          # SPY absent des 300 premières dates
prices.iloc[1000:1003, 1] = np.nan    # trou de 3 jours sur EFA

rets = compute_returns(clean_prices(prices))
print(rets.isna().sum().head(3))

SPY    300
EFA      0
EEM      0
```

Le trou d'EFA est comblé; les dates où SPY n'existait pas restent en NaN, et le backtester n'investira dans SPY qu'une fois son historique complet.

3.4 Changer de fréquence

```
from portopt.data import resample_returns

W = resample_returns(R, "W-FRI") # hebdomadaire, fin de semaine le vendredi
M = resample_returns(R, "ME")    # mensuel, fin de mois
print(W.shape, M.shape)

(757, 10) (174, 10)
```

Attention

En hebdomadaire ou en mensuel, pensez à passer `periods_per_year=52` ou `12` aux fonctions de `metric` et à `annualized_turnover` (section 6.3).

4 Module estimator

4.1 Interface commune

Toutes les règles d'allocation suivent la même interface, inspirée de scikit-learn :

```
import numpy as np
import pandas as pd
from portopt import ERC, risk_contributions

window = R.iloc[-252:] # une fenêtre d'un an
est = ERC(cov_method="ledoit_wolf").fit(window)
print(est.weights_.round(3))
```

```
SPY    0.073
EFA    0.064
EEM    0.048
IWM    0.060
TLT    0.145
IEF    0.251
LQD    0.146
GLD    0.089
DBC    0.071
VNQ    0.054
Name: ERC, dtype: float64
```

```
print(risk_contributions(est.weights_, est.cov_).round(3))

[0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1]
```

Élément	Description
<code>fit(returns)</code>	Calcule les poids sur une fenêtre sans NaN; renvoie l'estimateur.
<code>weights_</code>	Poids (<code>pd.Series</code> , somme 1, positifs). Son nom est celui de la classe.
<code>cov_</code>	Covariance utilisée (<code>ndarray</code>), pour les règles qui en estiment une.
<code>name</code>	Nom de la classe, utilisé comme libellé par défaut.
<code>cov_method</code>	"sample", "ledoit_wolf", "ewma" ou une fonction <code>returns -> ndarray</code> .

4.2 Règles disponibles

Classe	Paramètres propres
EqualWeight()	aucun; $w_i = 1/N$.
InverseVolatility(exponent=1.0)	$w_i \propto \sigma_i^{-k}$; <code>exponent=2</code> pour l'inverse variance.
RiskParity(budgets=None, tol=1e-10, max_iter=10000)	budgets de risque : dict, liste ou <code>pd.Series</code> , un par actif, strictement positifs, normalisés automatiquement. <code>None</code> donne l'ERC. Attribut <code>n_iter_</code> .
ERC(tol, max_iter)	RiskParity avec budgets égaux.
HRP(linkage="single", split="bisection")	<code>linkage</code> : "single", "complete", "average", "ward"; <code>split</code> : "bisection" (article original) ou "dendrogram". Attributs <code>order_</code> et <code>linkage_matrix_</code> .

4.3 Budgets de risque

```
from portopt import RiskParity

budgets = {t: 1 for t in R.columns} | {"TLT": 2, "IEF": 2, "LQD": 2}
rb = RiskParity(budgets=budgets).fit(window)
rc = risk_contributions(rb.weights_, rb.cov_)
print(pd.Series(rc, index=R.columns).round(3))
```

```
SPY    0.077
EFA    0.077
EEM    0.077
IWM    0.077
TLT    0.154
IEF    0.154
LQD    0.154
GLD    0.077
DBC    0.077
VNQ    0.077
```

Les budgets 2 et 1 sont normalisés : les trois ETF obligataires portent chacun $2/13 \approx 15.4\%$ du risque. Un budget manquant pour un actif de la fenêtre lève `ValueError: budgets missing for some assets of the window`.

4.4 HRP

```
from portopt import HRP

h = HRP(linkage="ward", split="dendrogram").fit(window)
print(h.order_)

['LQD', 'TLT', 'IEF', 'VNQ', 'SPY', 'IWM', 'EFA', 'EEM', 'GLD', 'DBC']
```

`order_` donne l'ordre des feuilles de l'arbre, et `linkage_matrix_` l'arbre lui-même, que SciPy sait tracer :

```
from scipy.cluster.hierarchy import dendrogram
dendrogram(h.linkage_matrix_, labels=list(window.columns))
```

4.5 Choisir l'estimateur de covariance

Toute règle accepte une fonction à la place d'un nom. Par exemple, une covariance diagonale (corrélations ignorées) transforme l'ERC en inverse volatilité, ce qu'on peut vérifier :

```
from portopt import InverseVolatility

diag_cov = lambda r: np.diag(r.var().to_numpy())
w1 = ERC(cov_method=diag_cov).fit(window).weights_
w2 = InverseVolatility().fit(window).weights_
print(np.abs(w1 - w2).max())

8.326672684688674e-17
```

Les estimateurs sont aussi disponibles directement :

```
from portopt import estimate_cov, ledoit_wolf_cov
from portopt.estimator import ewma_cov

S = estimate_cov(window, "sample")
Sigma, delta = ledoit_wolf_cov(window) # delta : intensité du shrinkage
S_ewma = ewma_cov(window, halflife=63)
```

4.6 Écrire sa propre règle

Il suffit d'hériter de `BaseEstimator` et d'implémenter `_compute_weights`, qui reçoit une fenêtre sans NaN et renvoie un vecteur de poids. `fit` met à zéro les poids négatifs et normalise la somme à 1. Exemple : le portefeuille de variance minimale long-only.

```
from scipy.optimize import minimize
from portopt import BaseEstimator

class MinVariance(BaseEstimator):
    def _compute_weights(self, returns):
        cov = self._cov(returns) # applique cov_method
        cov = cov / np.trace(cov) # mise à l'échelle, même solution
        n = cov.shape[0]
        res = minimize(lambda w: w @ cov @ w, np.full(n, 1 / n),
                        jac=lambda w: 2 * cov @ w, method="SLSQP",
                        bounds=[(0, 1)] * n,
                        constraints={"type": "eq", "fun": lambda w: w.sum() - 1},
                        options={"ftol": 1e-12, "maxiter": 500})

        return res.x

print(MinVariance(cov_method="ledoit_wolf").fit(window).weights_.round(3))
```

```
SPY    0.130
EFA    0.020
EEM    0.000
IWM    0.014
TLT    0.000
IEF    0.611
LQD    0.155
GLD    0.020
DBC    0.051
VNQ    0.000
Name: MinVariance, dtype: float64
```

Attention

Sans la mise à l'échelle et le gradient explicite, SLSQP s'arrête dès le premier pas : les variances quotidiennes sont de l'ordre de 10^{-5} , et la fonction objectif passe sous sa tolérance par défaut. L'optimiseur renvoie alors exactement $1/N$, sans message d'erreur. Normaliser la covariance ne change pas la solution.

5 Module backtester

5.1 Un backtest

```
from portopt import Backtester

bt = Backtester(R, ERC(), in_sample=252, out_of_sample=21, transaction_cost_bps=5)
r = bt.run()
print(type(r).__name__, r.shape, r.index[0].date(), r.name)
```

Series (3528,) 2010-12-21 ERC

Les rendements commencent après la première fenêtre de 252 dates. Chaque date de rebalancement utilise uniquement les données antérieures.

Paramètre	Rôle
returns	rendements (DataFrame), NaN autorisés.
estimator	règle d'allocation ; copiée à chaque rebalancement.
in_sample	longueur de la fenêtre d'estimation L (au moins 2, et inférieure au nombre de dates).
out_of_sample	période de détention H entre deux rebalancements.
window	"rolling" (fenêtre glissante de L dates) ou "expanding" (depuis la première date).
transaction_cost_bps	coût proportionnel au turnover, en points de base.
drift	True : les poids dérivent avec les prix entre deux rebalancements (réaliste). False : rebalancement quotidien gratuit vers la cible.
min_assets	nombre minimal d'actifs éligibles pour rebalancer ; sinon le portefeuille précédent est conservé.

5.2 Ce que renvoie le backtest

```
print(bt.weights_.iloc[:3, :4].round(3))
print(bt.turnover_.iloc[:3].round(3))
print(round(bt.annualized_turnover(), 3))
```

```

      SPY    EFA    EEM    IWM
2010-12-21  0.079  0.074  0.057  0.066
2011-01-19  0.078  0.070  0.053  0.065
2011-02-17  0.073  0.072  0.055  0.062

2010-12-21    1.000
2011-01-19    0.037
2011-02-17    0.027

0.488
```

Élément	Description
<code>run()</code>	lance le backtest et renvoie les rendements nets de coûts (<code>pd.Series</code>).
<code>returns_</code> , <code>gross_returns_</code>	rendements nets et bruts de coûts.
<code>weights_</code>	poids cibles à chaque rebalancement (<code>DataFrame</code> dates × actifs).
<code>turnover_</code>	turnover one-way à chaque rebalancement ; le premier vaut 1 (départ en cash).
<code>annualized_turnover(periods_per_year=252)</code>	turnover moyen par an, hors construction initiale.
<code>risk_contributions(cov_method="sample")</code>	contributions au risque ex ante à chaque rebalancement.

5.3 Actifs cotés en cours de période

Un actif dont la fenêtre contient des NaN reçoit un poids nul, puis entre dans le portefeuille dès qu'il a L dates complètes. Aucun traitement n'est à faire en amont :

```
R2 = R.copy()
R2.iloc[:600, 0] = np.nan # SPY coté à partir de la date 600
bt2 = Backtester(R2, HRP(), 252, 21)
bt2.run()
print(bt2.weights_["SPY"].iloc[0], round(bt2.weights_["SPY"].iloc[-1], 3))
```

0.0 0.02

Pendant la détention, un NaN est traité comme un rendement nul (prix figé).

5.4 Comparer plusieurs règles

```
from portopt import EqualWeight, run_backtests, summary

rules = {"1/N": EqualWeight(), "ERC": ERC(), "HRP": HRP()}
rets, bts = run_backtests(R, rules, 252, 21, transaction_cost_bps=5)
print(rets.shape, list(rets.columns))
```

(3528, 3) ['1/N', 'ERC', 'HRP']

`run_backtests` applique la même grille à chaque règle et renvoie les rendements (`DataFrame`, une colonne par règle) et le dictionnaire des `Backtester`. Les paramètres supplémentaires (`window`, `drift`, ...) sont transmis à chaque backtest.

Attention

Passez un **dictionnaire** {libellé: estimateur}. Une liste est acceptée, mais les libellés sont alors les noms de classe : `[ERC(), ERC(cov_method="ledoit_wolf")]` lève une erreur, car les deux s'appelleraient `ERC`.

6 Module metric

6.1 Le tableau de synthèse

```
turnover = {k: b.annualized_turnover() for k, b in bts.items()}
print(summary(rets, turnover=turnover).round(3))
```

	1/N	ERC	HRP
CAGR	0.069	0.062	0.061

Ann. mean	0.074	0.063	0.062
Ann. vol	0.114	0.082	0.070
Sharpe	0.644	0.772	0.879
Sortino	0.925	1.140	1.329
Calmar	0.254	0.325	0.408
Max DD	-0.273	-0.190	-0.150
Max DD duration	923.000	760.000	426.000
VaR 95%	0.010	0.008	0.006
CVaR 95%	0.017	0.012	0.010
Skew	0.195	0.648	0.893
Excess kurt	13.689	16.898	16.310
Hit ratio	0.535	0.540	0.534
PSR(SR>0)	0.992	0.998	1.000
Ann. turnover	0.383	0.488	1.203

`summary(returns, rf=0.0, periods_per_year=252, alpha=0.05, turnover=None)` accepte une **Series** ou un **DataFrame**. La durée du drawdown est en nombre de périodes ; VaR et CVaR sont des pertes positives sur une période.

6.2 Les fonctions

Toutes acceptent une **Series** (renvoient un nombre) ou un **DataFrame** (renvoient une **Series**, une valeur par colonne).

Fonction	Résultat
<code>total_return, cagr</code>	rendement cumulé, taux annuel composé
<code>annualized_mean, annualized_volatility</code>	moyenne $\times P$, écart-type $\times \sqrt{P}$
<code>sharpe_ratio(r, rf=0.0)</code>	Sharpe annualisé ; <code>rf</code> annuel
<code>sortino_ratio(r, target=0.0)</code>	Sortino annualisé ; <code>target</code> par période
<code>calmar_ratio</code>	CAGR divisé par la valeur absolue du drawdown maximal
<code>wealth_index, drawdown_series</code>	courbe de richesse, drawdown à chaque date
<code>max_drawdown, max_drawdown_duration</code>	drawdown maximal (négatif), durée sous l'eau
<code>value_at_risk(r, alpha=0.05), conditional_value_at_risk</code>	VaR et CVaR historiques
<code>skewness, excess_kurtosis, hit_ratio</code>	asymétrie, kurtosis excédentaire, part de rendements positifs
<code>probabilistic_sharpe_ratio(r, sr_benchmark=0.0)</code>	probabilité que le vrai Sharpe dépasse le seuil (annualisé)

```
from portopt import metric as M

x = rets["ERC"]
print(round(M.sharpe_ratio(x), 3), round(M.sharpe_ratio(x, rf=0.03), 3))
print(round(M.probabilistic_sharpe_ratio(x), 3),
      round(M.probabilistic_sharpe_ratio(x, sr_benchmark=0.5), 3))
```

0.772 0.412
0.998 0.848

Avec un taux sans risque de 3 %, le Sharpe d'ERC tombe de 0.77 à 0.41. La probabilité que son vrai Sharpe dépasse 0.5 n'est que de 85 %.

6.3 Autres fréquences

```
W = resample_returns(R, "W-FRI")
wr, wbps = run_backtests(W, {"ERC": ERC()}, in_sample=52, out_of_sample=4)
tbl = summary(wr, periods_per_year=52)
print(tbl.loc[["CAGR", "Ann. vol", "Sharpe"]].round(3))
print(round(wbps["ERC"].annualized_turnover(periods_per_year=52), 3))
```

	ERC
CAGR	0.063
Ann. vol	0.083
Sharpe	0.783
	0.74

`in_sample` et `out_of_sample` sont exprimés en nombre de périodes : 52 semaines et 4 semaines ici.

7 Module plot

7.1 Le rapport complet

```
from portopt import plot

paths = plot.save_report(rets, bts, "outputs/charts")
print([p.name for p in paths])

['01_wealth.png', '02_drawdowns.png', '03_rolling_vol.png', '04_rolling_sharpe.png',
 '05_metrics.png', '06_avg_weights.png', '07_avg_risk_contrib.png',
 '08_weights_1_n.png', '08_weights_erc.png', '08_weights_hrp.png']
```

`save_report` produit sept graphiques de comparaison et un graphique de poids par stratégie. Chaque stratégie garde la même couleur dans tous les graphiques.

Drawdowns

Peak-to-trough loss of wealth, dot = maximum drawdown

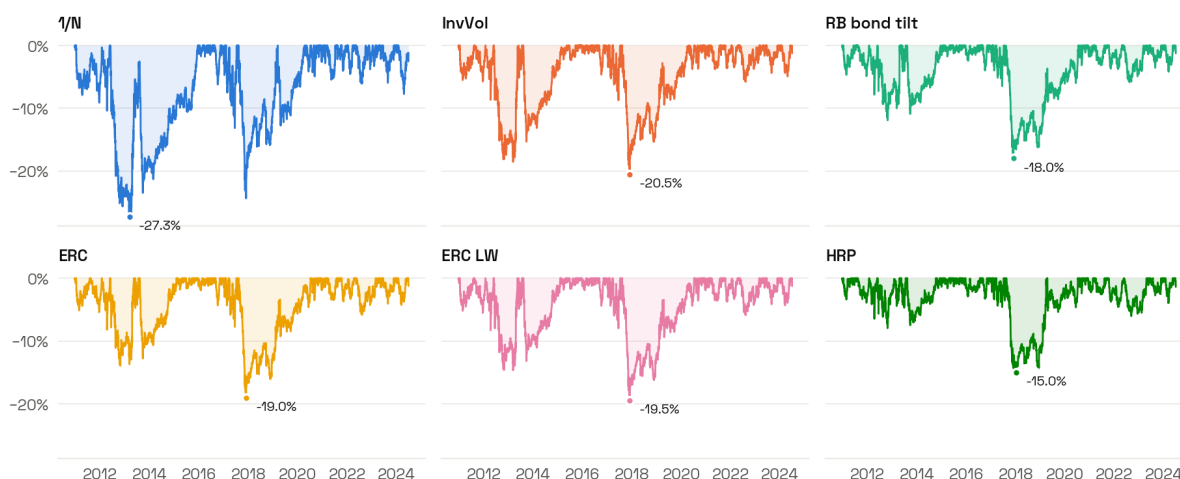


Fig. 1. `plot_drawdowns` : un panneau par stratégie, drawdown maximal annoté.

7.2 Les fonctions

Fonction	Graphique
<code>use_style()</code>	active le style (fond blanc, polices Space Grotesk); appelé par <code>save_report</code> .
<code>plot_wealth(rets, log=True)</code>	croissance de 1, avec la valeur finale au bout de chaque courbe.
<code>plot_drawdowns(rets)</code>	drawdowns, un panneau par stratégie.
<code>plot_rolling(rets, stat="vol", window=252)</code>	volatilité ou Sharpe (<code>stat="sharpe"</code>) glissants.
<code>plot_metrics(summary_df, metrics=None)</code>	barres horizontales, un panneau par indicateur.
<code>plot_heatmap(df, title)</code>	tableau coloré annoté (poids, contributions au risque).
<code>plot_weights_through_time(bt.weights_, title)</code>	poids de chaque actif à chaque rebalancement.
<code>plot_correlation(R, order=None)</code>	matrice de corrélation, dans l'ordre souhaité (par exemple <code>h.order_</code>).
<code>colors_for(names)</code>	couleur attribuée à chaque stratégie.

Chaque fonction renvoie un objet matplotlib (Axes ou Figure) que l'on peut modifier puis enregistrer :

```
plot.use_style()
ax = plot.plot_wealth(rets)
ax.figure.savefig("wealth.png")
```

Attention

`plot_rolling` et `save_report` supposent des données quotidiennes (annualisation sur 252 périodes). Pour des données hebdomadaires ou mensuelles, calculez les indicateurs avec `summary(..., periods_per_year=...)` et utilisez les autres fonctions de tracé.

8 Recettes

8.1 Comparaison complète en ligne de commande

```
python examples/run_backtest.py # 10 ETF, Yahoo Finance, depuis 2008
python examples/run_backtest.py --synthetic # univers simulé, sans réseau
python examples/run_backtest.py --in-sample 504 --out-of-sample 63 \
    --tc-bps 10
```

Le script compare six règles (1/N, InvVol, risk budgeting orienté obligations, ERC, ERC Ledoit-Wolf, HRP) et écrit dans `outputs/` : `summary.csv`, `oos_returns.csv` et le rapport graphique dans `outputs/charts/`. Options : `--start`, `--end`, `--in-sample`, `--out-of-sample`, `--tc-bps`, `--out`.

8.2 Tester la robustesse aux paramètres

La fenêtre L et la période de détention H sont des choix arbitraires. Un résultat fiable doit résister à leur variation : on regarde le classement des règles sur une grille, pas le meilleur couple.

```
grid = {}
for L in (126, 252, 504):
    for H in (5, 21, 63):
```

```

out, _ = run_backtests(R, rules, L, H, transaction_cost_bps=5)
grid[(L, H)] = summary(out).loc["Sharpe"].rank(ascending=False).astype(int)
print(pd.DataFrame(grid).T)

```

		1/N	ERC	HRP
126	5	3	2	1
	21	3	2	1
	63	3	2	1
252	5	3	2	1
	21	3	2	1
	63	3	2	1
504	5	3	1	2
	21	3	1	2
	63	3	2	1

1/N est dernier partout ; l'ordre entre ERC et HRP dépend de la fenêtre.

8.3 Mesurer la part de hasard du calendrier

Décaler la date de départ change les dates de rebalancement. L'écart obtenu mesure ce qui relève du hasard du calendrier (*timing luck*) :

```

luck = {}
for offset in range(0, 21, 5):
    out, _ = run_backtests(R.iloc[offset:], {"ERC": ERC()}, 252, 21)
    luck[offset] = summary(out).loc["Sharpe", "ERC"]
print(pd.Series(luck).round(3))

```

0	0.776
5	0.787
10	0.782
15	0.777
20	0.782

Ici l'écart est faible (0.78 ± 0.01) : le résultat d'ERC ne dépend pas du calendrier.

8.4 Exporter les résultats

```

rets.to_csv("oos_returns.csv")
summary(rets).to_csv("summary.csv")
bts["ERC"].weights_.to_csv("erc_weights.csv")
bts["ERC"].risk_contributions().to_csv("erc_risk_contrib.csv")

```

8.5 Régénérer la documentation

python docs/make_figures.py	# figures de la méthodologie
cd docs && latexmk -xelatex methodology.tex	# méthodologie (FR)
latexmk -xelatex methodology_en.tex	# méthodologie (EN)
latexmk -xelatex user_guide.tex	# ce guide

9 Erreurs fréquentes

Message	Cause et solution
returns contain NaN: select a clean universe before fitting.	fit appelé directement sur une fenêtre avec des NaN. Sélectionner les actifs complets (<code>window.dropna(axis=1)</code>) ou passer par le Backtester, qui le fait seul.

Message	Cause et solution
<code>budgets missing for some assets of the window.</code>	<code>RiskParity</code> : un actif de la fenêtre n'a pas de budget. Donner un budget à chaque actif.
<code>budgets must be strictly positive (log barrier).</code>	Budget nul ou négatif. Pour exclure un actif, le retirer des rendements.
<code>in_sample (...) >= number of observations (...).</code>	Fenêtre plus longue que l'historique. Réduire <code>in_sample</code> ou allonger l'historique.
<code>call run() first.</code>	<code>annualized_turnover()</code> ou <code>risk_contributions()</code> avant <code>run()</code> .
<code>Several estimators share the name [...]</code>	Liste passée à <code>run_backtests</code> avec deux règles de même classe. Utiliser un dictionnaire.
<code>No data returned by Yahoo Finance for [...]</code>	Pas de réseau, ticker inconnu ou période vide. Vérifier les tickers ; travailler avec <code>cache_dir</code> ou <code>simulate_returns()</code> .
<code>No data for tickers: [...]</code>	Certains tickers n'ont renvoyé aucun prix. Les retirer de la liste.
<code>Field '...' not in [...]</code>	Champ absent : avec <code>auto_adjust=True</code> , Yahoo ne renvoie plus <code>Adj Close</code> ; utiliser <code>field="Close"</code> , déjà ajusté.
Poids d'une règle personnalisée égaux à 1/N	Optimiseur arrêté dès le départ à cause de l'échelle des variances (section 4). Normaliser la covariance et fournir le gradient.
Indicateurs aberrants en hebdomadaire	<code>periods_per_year</code> laissé à 252. Passer 52 (ou 12 en mensuel).

10 Référence de l'API

portopt.data

```
download_prices(tickers, start=None, end=None, field="Close", auto_adjust=True,
                cache_dir=None) -> DataFrame
clean_prices(prices, ffill_limit=5, min_history=None) -> DataFrame
compute_returns(prices, method="simple", period=1) -> DataFrame | Series
resample_returns(returns, rule="W-FRI", method="simple") -> DataFrame | Series
load_returns(tickers, start=None, end=None, method="simple", cache_dir=None,
             ffill_limit=5) -> DataFrame
simulate_returns(n_obs=3780, start="2010-01-01", seed=15) -> DataFrame
```

portopt.estimator

```
EqualWeight(cov_method="sample")
InverseVolatility(cov_method="sample", exponent=1.0)
RiskParity(cov_method="sample", budgets=None, tol=1e-10, max_iter=10000)
ERC(cov_method="sample", tol=1e-10, max_iter=10000)
HRP(cov_method="sample", linkage="single", split="bisection")
BaseEstimator.fit(returns) -> self # attributs weights_, cov_
estimate_cov(returns, method="sample") -> ndarray
sample_cov(returns) -> ndarray
ledoit_wolf_cov(returns) -> (ndarray, float)
ewma_cov(returns, halflife=63.0) -> ndarray
risk_contributions(weights, cov) -> ndarray
```

portopt.backtester

```
Backtester(returns, estimator, in_sample, out_of_sample, window="rolling",
            transaction_cost_bps=0.0, drift=True, min_assets=1)
.run() -> Series
.annualized_turnover(periods_per_year=252) -> float
```

```
.risk_contributions(cov_method="sample") -> DataFrame
run_backtests(returns, estimators, in_sample, out_of_sample, **kwargs)
-> (DataFrame, dict[str, Backtester])
```

portopt.metric

```
summary(returns, rf=0.0, periods_per_year=252, alpha=0.05, turnover=None) -> DataFrame
total_return(r)
cagr(r, periods_per_year=252)
annualized_mean(r, periods_per_year=252)
annualized_volatility(r, periods_per_year=252)
sharpe_ratio(r, rf=0.0, periods_per_year=252)
sortino_ratio(r, target=0.0, periods_per_year=252)
calmar_ratio(r, periods_per_year=252)
wealth_index(r, start=1.0)
drawdown_series(r)
max_drawdown(r)
max_drawdown_duration(r)
value_at_risk(r, alpha=0.05)
conditional_value_at_risk(r, alpha=0.05)
skewness(r)
excess_kurtosis(r)
hit_ratio(r)
probabilistic_sharpe_ratio(r, sr_benchmark=0.0, periods_per_year=252)
```

portopt.plot

```
use_style() colors_for(names)
plot_wealth(returns, ax=None, log=True, colors=None, title="Growth of 1")
plot_drawdowns(returns, colors=None, ncols=3)
plot_rolling(returns, stat="vol", window=252, colors=None, ax=None)
plot_metrics(summary, metrics=None, colors=None, ncols=3)
plot_heatmap(values, title, subtitle="", fmt="{:.0%}", diverging=False, ax=None,
              vmax=None, annot_size=8.5)
plot_weights_through_time(weights, title, ax=None)
plot_correlation(returns, order=None, title="Correlation matrix", subtitle="", ax=None)
save_report(returns, backtesters, out_dir, rolling_window=252) -> list[Path]
```